

Network Traffic Inspection & Rogue Service Detection System

Information Security Lab — Final Project Report
4th Semester



Submitted To:

Sonia Asghar

Submitted By:

Nouman Ahmed 2024-CS-740

Abshar Hussain 2024-CS-742

Daniyal Haider 2024-CS-746

M.Tahir 2024-CS-704

1. Introduction

This project was built for the Information Security (IS) Lab as a practical demonstration of the CIA Triad — Confidentiality, Integrity, and Availability. Instead of just reading about these concepts, we actually built a working system that shows what happens when security is missing and what happens when it is properly applied.

The project is split into two main parts. Module A deals with network traffic — it shows how a password typed into a plain HTTP website can be read by anyone on the network, and then demonstrates how switching to HTTPS completely hides that same password. Module B is a network scanner and auditor — it automatically scans a network, finds every open service running on machines, and raises an alert if any dangerous or unauthorized service is found.

Everything runs from a single, polished terminal dashboard called NITDS (Network Inspection and Threat Detection System). The dashboard was built using Python and the 'rich' library, giving it a clean, colour-coded interface with live status indicators.

1.1 Objectives

- Show the real danger of sending passwords over plain HTTP (no encryption).
- Show how TLS/HTTPS fixes that problem — the sniffer captures nothing useful.
- Build an automated network scanner that finds all open ports and services.
- Create a policy-based engine that automatically flags dangerous services as ROGUE.
- Package everything in one easy-to-use terminal dashboard.

1.2 Tools and Technologies Used

Tool/Library	Purpose
Python 3.10+	Core programming language for all modules
rich	Terminal UI library for the dashboard (colours, tables, panels)
Python http.server	Built-in module used to run the HTTP and HTTPS servers
ssl (Python stdlib)	Wraps the server socket with TLS encryption
OpenSSL	Generates the self-signed X.509 certificate and RSA private key
pyshark / tshark	Packet capture and dissection (Wireshark's command-line engine)
nmap	Port scanner used by the network auditor to find open services
policy.json	JSON file that defines which ports are safe and which are rogue

2. System Architecture

The project is organized into two modules, each living in its own folder, and a single controller script that ties everything together.

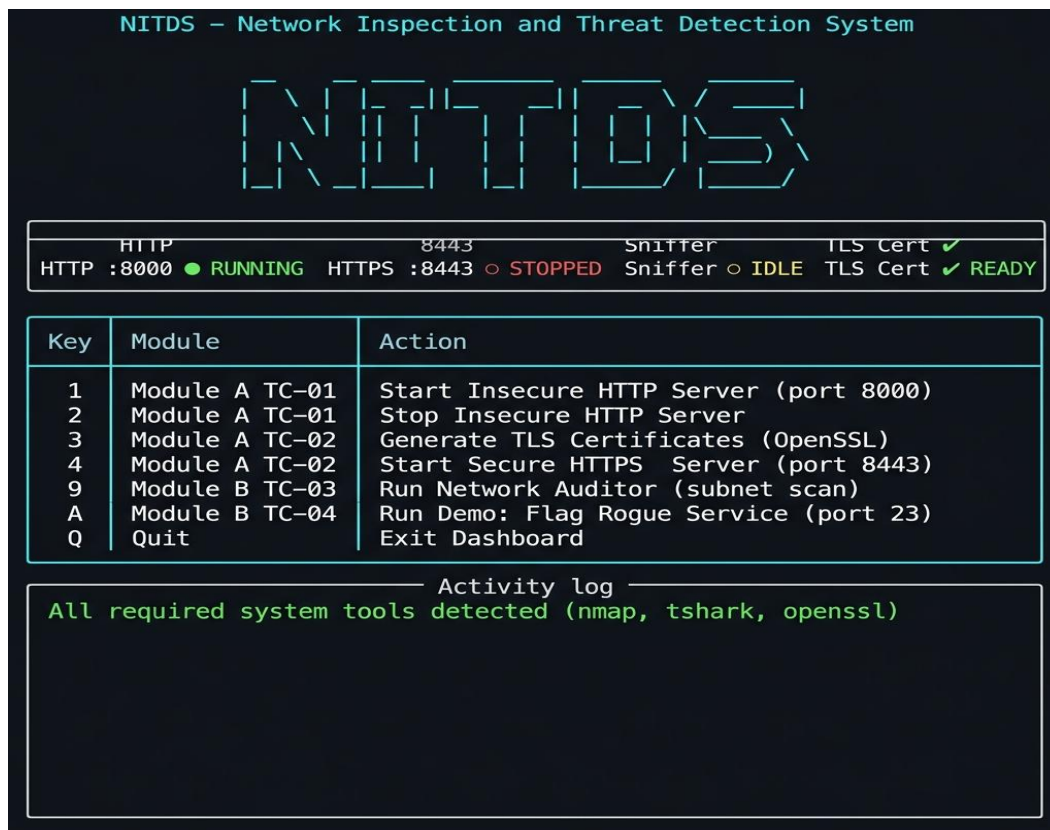
2.1 Project Structure

```
is_lab_project/
├── main_controller.py      ← Central TUI dashboard (entry point)
├── requirements.txt        ← pip dependencies
├── module_a_traffic/
│   ├── insecure_server.py  ← TC-01: Plain HTTP server (port 8000)
│   ├── secure_server.py    ← TC-02: TLS-encrypted HTTPS server (port 8443)
│   ├── generate_certs.sh   ← Creates server.crt and server.key via OpenSSL
│   └── sniffer_agent.py    ← Captures packets; reads creds (HTTP) or verifies
├── TLS (HTTPS)
└── module_b_detector/
    ├── network_auditor.py  ← Nmap engine + policy-based rogue detector
    └── policy.json          ← Authorized vs rogue port definitions
```

2.2 Main Dashboard (main_controller.py)

The `main_controller.py` file is the front door of the entire project. When you run it, you see a full-screen terminal interface with a banner, a live status bar, a menu, and a rolling activity log. You never need to open individual scripts manually — every action in the project is triggered from this one dashboard. The dashboard tracks the state of all running processes (HTTP server, HTTPS server, sniffer) and shows whether they are running or stopped in real time. It uses Python's `subprocess` module to launch each script as a background process, reading its output and feeding it into the activity log.

Figure 1: NITDS Main Dashboard — status bar, menu, and activity log



3. Module A — Network Traffic Inspection

Module A has one core message: if you send data over plain HTTP, anyone on the network can read it. If you send it over HTTPS, they cannot. We prove this in two test cases.

3.1 Test Case TC-01 — Insecure HTTP Credential Capture

Security Concept: Violation of Confidentiality

`insecure_server.py` starts a web server on port 8000 that serves a realistic-looking Instagram login page. The page is built with full CSS styling so it looks genuine. When a user types their username and password and clicks Log In, the browser sends a POST request to `/login`. Because there is no encryption, this POST request travels through the network as plain readable text. At the same time, `sniffer_agent.py` is running in a separate process. It uses `pyshark` (a Python wrapper around `tshark`, which is Wireshark's command-line version) to listen on the loopback network interface. When the POST request arrives, `pyshark` reads the `http.file_data` field of the packet — which contains the form body — and extracts the username and password directly from it.

How to Run TC-01

```
# Terminal 1 - start the HTTP server:
python3 module_a_traffic/insecure_server.py
# Terminal 2 - start the sniffer (needs root for packet capture):
sudo python3 module_a_traffic/sniffer_agent.py --mode http --interface lo
# Terminal 3 - submit a login:
curl -d 'username=alice&password=secret123' http://localhost:8000/login
```

TC-01 Expected Output

The sniffer immediately prints the captured credentials. The username 'alice' and password 'secret123' appear in full on the terminal — exactly as typed by the user. This proves that without encryption, a passive attacker on the same network can silently steal credentials without the victim

```
MODULE A — SNIFFER AGENT
Mode:      HTTP (Plaintext)
Interface: lo
Pkt Limit: 100

[pyshark] Starting HTTP capture on interface 'lo' port 8000...

Submit the login form now (http://localhost:8000)

[!!] PLAINTEXT CREDENTIALS EXTRACTED FROM WIRE
Packet #47
Username : alice
Password : secret123
Source   : 127.0.0.1:52840
Dest     : 127.0.0.1:8000

[!!] These are IDENTICAL to what tshark captures on wire!
```

knowing anything happened.

3.2 TLS Certificate Generation

Before we can run the secure server, we need a TLS certificate. The file `generate_certs.sh` uses OpenSSL to create a self-signed X.509 certificate and a 2048-bit RSA private key. The command used is:

```
openssl req -x509 -newkey rsa:2048 -nodes -days 365 \  
-subj '/C=PK/ST=Punjab/L=Lahore/O=IS-Lab-Demo/OU=Security/CN=localhost' \  
-keyout server.key -out server.crt
```

This creates two files: `server.key` (the private key, kept secret on the server) and `server.crt` (the public certificate, shared with clients during the TLS handshake). In a real-world scenario, this certificate would be signed by a trusted Certificate Authority like Let's Encrypt. For this lab, a self-signed certificate is enough to enable and test TLS.

MODULE A – OpenSSL Certificate Generation

Generating 2048-bit RSA private key...

Output: `/path/to/module_a_traffic/server.key`

Output: `/path/to/module_a_traffic/server.crt`

Generating a RSA private key

.....++

..++

writing RSA key

✓ Certificate generated successfully!

Certificate Details:

Subject: C=PK, ST=Punjab, L=Lahore, O=IS-Lab-Demo, OU=Security, CN=localhost

Issuer: C=PK, ST=Punjab, L=Lahore, O=IS-Lab-Demo, OU=Security, CN=localhost

Not Before: Jun 7 05:48:13 2026 GMT

Not After : Jun 7 05:48:13 2027 GMT

Public-Key: (2048 bit)

You can now start the secure server: `python3 secure_server.py`

Figure 3: OpenSSL generating the RSA private key and self-signed certificate

3.3 Test Case TC-02 — HTTPS TLS Verification

Security Concept: Confidentiality via TLS

secure_server.py runs the exact same login page but on port 8443 with TLS enabled. Internally, Python's ssl.SSLContext wraps the raw TCP socket with TLS. Every byte that leaves the server is encrypted before it hits the network. The browser and server go through a TLS handshake first, after which all communication is encrypted.

We run sniffer_agent.py again, this time in HTTPS mode. The sniffer still captures every packet on the wire. But now, instead of seeing form fields and passwords, it only sees TLS record headers — CLIENT HELLO, SERVER HELLO, CERTIFICATE, APPLICATION DATA. The APPLICATION DATA packets, which contain the actual credentials, are fully encrypted. The sniffer can see that a packet exists, but cannot read its contents.

How to Run TC-02

```
# Step 1 - generate certificates (only needed once):
bash module_a_traffic/generate_certs.sh
# Terminal 1 - start the HTTPS server:
python3 module_a_traffic/secure_server.py
# Terminal 2 - start the sniffer in HTTPS mode:
sudo python3 module_a_traffic/sniffer_agent.py --mode https --interface lo
# Terminal 3 - send a login:
curl -k -d 'username=alice&password=secret123' https://localhost:8443/login
```

TC-02 Expected Output

The sniffer output shows packets labelled Handshake, Change Cipher Spec, and Application Data (ENCRYPTED). Notice that the username and password never appear anywhere in the output. The final summary confirms: Readable credentials: NONE. This is TLS working exactly as it should.

```
MODULE A — SNIFFER AGENT
Mode: HTTPS (TLS Verification), Interface: lo, Pkt Limit: 100

[pyshark] Capturing HTTPS traffic on 'lo' port 8443...

[TLS] Pkt # 1 Record: Handshake      Handshake: CLIENT HELLO
[TLS] Pkt # 2 Record: Handshake      Handshake: SERVER HELLO
[TLS] Pkt # 3 Record: Handshake      Handshake: CERTIFICATE
[TLS] Pkt # 4 Record: Handshake      Handshake: SERVER HELLO DONE
[TLS] Pkt # 5 Record: Handshake      Handshake: CLIENT KEY EXCHANGE
[TLS] Pkt # 6 Record: Change Cipher Spec
[TLS] Pkt # 7 Record: Application Data (ENCRYPTED)
└─ Application data is ENCRYPTED — credentials NOT readable!

TC-02 RESULT: TLS Verification Summary
Total packets captured: 7
TLS records observed: 7
Readable credentials: NONE (fully encrypted)
```

4. Module B — Rogue Service Detection

Module B addresses a different security problem: how do you know what is actually running on your network? Attackers often install backdoors or leave dangerous legacy services running. Module B automates the discovery and flagging of such services.

The module has two components: `network_auditor.py` (the scanner and analyser) and `policy.json` (the rulebook that defines what is safe and what is not).

4.1 The Security Policy (`policy.json`)

`policy.json` is a simple JSON file that lists two categories of ports. Authorized ports are services that are known, approved, and secure. Rogue ports are services that should never be running — either because they send data in plaintext (like Telnet and FTP) or because they have no authentication.

Port	Service	Status	Risk	Reason
22	SSH	AUTHORIZED	None	Encrypted remote shell — approved
443	HTTPS	AUTHORIZED	None	Encrypted web — approved
8443	HTTPS-alt	AUTHORIZED	None	Lab HTTPS server — approved
53	DNS	AUTHORIZED	None	Name resolution — approved
21	FTP	ROGUE	HIGH	Cleartext file transfer
23	Telnet	ROGUE	CRITICAL	Cleartext remote shell
80	HTTP	ROGUE	HIGH	Unencrypted web traffic
8000	HTTP-dev	ROGUE	MEDIUM	Dev HTTP server, no encryption
110	POP3	ROGUE	HIGH	Cleartext email retrieval
69	TFTP	ROGUE	HIGH	No authentication, no encryption

4.2 How the Auditor Works

`network_auditor.py` works in five steps every time it runs:

- Step 1 — Load policy: reads `policy.json` to know which ports are authorized and which are rogue.
- Step 2 — Host discovery: runs 'nmap -sn' (ping scan) against the target subnet to find live machines.
- Step 3 — Service scan: for each live host, runs 'nmap -sV' to enumerate every open port and detect the software version running on it.
- Step 4 — Policy check: compares each open port against the policy. Ports in `authorized_ports` get a green SAFE label. Ports in `rogue_ports` get a red ROGUE label with risk level. Anything not in either list gets flagged as UNKNOWN for manual review.
- Step 5 — Report: prints a formatted per-host report and a final summary showing all critical findings.

4.3 Test Case TC-03 — Subnet Discovery and Port Listing

Security Concept: Threat Detection through Visibility

TC-03 demonstrates the basic audit flow. We scan the localhost (127.0.0.1) to see what services are currently running. The `--skip-discovery` flag tells nmap to skip the ping scan (not needed for a single known IP) and go straight to port scanning.

How to Run TC-03

```
python3 module_b_detector/network_auditor.py --subnet 127.0.0.1 --skip-discovery
```

```
# To scan a VM subnet instead:
```

```
python3 module_b_detector/network_auditor.py --subnet 192.168.56.0/24
```

```
MODULE B — NETWORK AUDITOR and ROGUE SERVICE DETECTOR

Target: 127.0.0.1
Started: 2026-06-07 10:45:22

[✓] Policy loaded: v1.0 — IS Lab Project — Network Security Demo
[✓] Nmap 7.80 ( https://nmap.org )
[INFO] Skipping discovery — scanning 127.0.0.1 directly.
[SCAN] Service version scan on 127.0.0.1...

Nmap command - 127.0.0.1 (4 open ports | 1 Network Security Demo —
nmap -7.80 0.1 8443 nmap -- Service discovery — scanning 127.0.0.1
127.0.0.1

Host: 127.0.0.1 (4 open ports | 1 ROGUE | 2 SAFE | 1 UNKNOWN)

✓ Port 22/tcp    ssh [OpenSSH 8.9]
✓ Port 8443/tcp  https-alt

✗ Port 8000/tcp  http-dev
  ⚠ HIGH RISK: Rogue Service Detected
  Risk  : MEDIUM
  Reason: Unauthenticated dev HTTP server — no encryption

? Port 9000/tcp  (unknown)
  ⚠ UNKNOWN: Port not in policy — requires manual review
```

Figure 5: TC-03 — Network audit showing safe ports and a flagged rogue HTTP server

The output shows each open port with a coloured label. Green checkmarks mean the service is authorized. Red X marks mean a rogue service was found. Port 8000 (our insecure HTTP server from TC-01) is correctly flagged as MEDIUM risk because it runs without encryption.

4.4 Test Case TC-04 — Rogue Service Flagging (Telnet on Port 23)

TC-04 simulates a real attack scenario: a rogue Telnet service has been started on the machine. Telnet is one of the most dangerous legacy protocols — it sends everything, including login credentials and all commands, in complete plaintext. Any attacker on the same network can hijack the entire session. The demo works in four automatic steps. First, the dashboard opens a netcat listener on port 23, simulating a rogue Telnet backdoor. Then it runs the network auditor against localhost. The auditor finds port 23 open and compares it against the policy, which marks Telnet as CRITICAL risk. A big red alert is printed. Finally, the rogue listener is automatically killed and cleaned up.

How to Run TC-04

```
# Use the dashboard (option A) which automates everything:
python3 main_controller.py → press A
# Or manually:
sudo nc -lvp 23 &
python3 module_b_detector/network_auditor.py --subnet 127.0.0.1 --skip-discovery
```

The output clearly shows the CRITICAL finding with the reason: Cleartext remote shell — complete session hijack possible. This is exactly what a real security audit would report. The auditor then prints a final summary listing every rogue service found, making it easy for a system administrator

TC-04 — Rogue Service Demo

This demo will:

1. Open a listener on port 23 (Telnet) using netcat,
2. Run the auditor against localhost,
3. Show the CRITICAL risk flag,
4. Kill the rogue listener

→ Starting rogue Telnet listener on port 23...

→ Running auditor to detect rogue service...

Host: 127.0.0.1 (3 open ports | 1 ROGUE | 2 SAFE | 0 UNKNOWN)

✓ ssh port 22

✓ https-alt port 8443

✗ Port 23/tcp telnet

△ HIGH RISK: Rogue Service Detected

Risk : CRITICAL

Reason: Cleartext remote shell — complete session hijack possible

CRITICAL FINDINGS — ROGUE SERVICES DETECTED:

[CRITICAL] 127.0.0.1:23/tcp (telnet) — Cleartext remote shell — complete session hijack possible

✓ Rogue listener cleaned up.

5. Security Concepts Demonstrated

CIA Principle	Where Demonstrated	How
Confidentiality (violated)	TC-01 — HTTP sniffer	Credentials captured in plaintext from unencrypted HTTP traffic
Confidentiality (enforced)	TC-02 — HTTPS with TLS	Same credentials become unreadable on the wire after TLS is applied
Integrity	TC-02 — TLS MAC/AEAD	TLS prevents any in-transit modification of packets
Identity / Authentication	TLS Certificate (OpenSSL)	Server certificate proves server identity to the client
Threat Detection	TC-03 — Nmap scan	Automated port scan reveals the full attack surface of a host
Vulnerability Assessment	TC-04 — Policy engine	Policy flags legacy cleartext protocols as HIGH or CRITICAL risk

6. Results Summary

Test Case	Module	Goal	Result
TC-01	A	Capture credentials over plain HTTP	PASS — username and password captured from wire in full plaintext
TC-02	A	Verify TLS hides credentials	PASS — only TLS handshake visible; no credentials readable
TC-03	B	Discover hosts and list open ports	PASS — all open ports listed with service names and versions
TC-04	B	Flag rogue service (Telnet port 23)	PASS — port 23 flagged CRITICAL; reason printed in report

Key Observations

- HTTP is fundamentally insecure for any login form. The sniffer required no special attack — it simply read packets that were already visible on the network.
- Adding TLS required only three things: a certificate, a private key, and wrapping the socket with `ssl.SSLContext`. The application code did not change at all.
- Automated network auditing provides a huge advantage. Instead of manually checking each machine, the auditor scans the entire subnet and flags every problem in seconds.
- The `policy.json` approach is flexible and realistic. In a production environment, security teams maintain exactly this kind of allowlist/denylist policy.

7. Conclusion

This project successfully built a working, hands-on demonstration of core information security concepts. Rather than just describing threats in theory, we built real tools that show the threat happening and then show the fix working.

Module A proved that plain HTTP offers zero protection for user credentials. Any passive listener on the network — whether a legitimate network admin, a lab tool, or a malicious attacker — can read usernames and passwords without any effort. Switching to HTTPS with even a self-signed certificate completely eliminated that risk. The sniffer went from capturing full credentials to capturing nothing of value.

Module B showed that visibility is the first step in security. You cannot protect what you do not know exists. The automated auditor, combined with a clear policy definition, makes it straightforward to find dangerous services like Telnet and FTP and raise an alert before an attacker can exploit them.

Together, these two modules form a practical mini security toolkit that reflects real-world practices: network traffic inspection, TLS enforcement, port scanning, and policy-based vulnerability assessment.

8. References

- Python Software Foundation. ssl — TLS/SSL wrapper for socket objects. <https://docs.python.org/3/library/ssl.html>
- Nmap Network Scanning — The Official Nmap Project Guide. <https://nmap.org/book/>
- RFC 5246 — The Transport Layer Security (TLS) Protocol Version 1.2
- RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3
- Wireshark / tshark Documentation. <https://www.wireshark.org/docs/>
- OpenSSL Documentation. <https://www.openssl.org/docs/>
- Willems, C. et al. (2012). Towards automated network security analysis. Security and Communication Networks.